

CAM-Resident Behavioral Monitoring for LLMs: Encoding Choices, Cross-Domain Fidelity, and Library-Size Scaling

Anonymized for review
DATE 2027 submission

Abstract—Frontier language model deployments increasingly treat a model’s own activations as a safety signal: a production jailbreak defense from Anthropic scores a single linear probe over residual-stream activations on every generated token, at roughly 10^{-3} percent of forward-pass compute. What is monitored is set by K , the number of stored behavioral signatures each activation is compared against, and target K ’s are increasing rapidly: from $K = 1$ in that defense, to 10^2 – 10^3 once probes are fitted per behavior, domain, and layer, to 1.3×10^5 per layer for a whole-vocabulary readout. On a GPU the signature bank streams from high-bandwidth memory once per monitored token, so at high enough K or token rates the monitor’s own memory traffic can exceed the bandwidth of the accelerator it runs on. We propose supporting this search with content-addressable memories (CAMs), which evaluate distance functions inside the memory array, trading arithmetic precision and distance-function fidelity for orders of magnitude less data movement. We quantify this trade on two workloads: (1) a ten-domain cross-domain deception grid over Llama-3.3-70B activations, where 384-bit codes hold in-domain AUROC within 0.01 of the full-precision cosine ceiling at more than $300\times$ compression per signature; and (2) whole-vocabulary readout, where a 3-bit Euclidean code at $\sim 10^3$ bits per row reproduces 0.9% of the exact readout’s top-1 selections. The whole-vocabulary bank then occupies 6–33 MB per layer, and its estimated in-array search energy is five to six orders of magnitude below streaming the equivalent bank from HBM.

Index Terms—Content-addressable memory; compute-in-memory; large language models; activation probes; AI safety monitoring; nearest-neighbor search; ferroelectric FETs; Hamming distance

I. INTRODUCTION

Large language model (LLM) deployments are wrapped in layered safeguards, and until recently every layer read *text* [1]. That has started to change: the always-on stage of one production jailbreak defense now reads the model’s activations directly [2], [3], as do deployed residual-stream probes for Gemini [4]. The deployed check is small – one stored vector, one dot product per generated token, one threshold – and it watches for one thing. In a recent cyber evaluation campaign, evaluators found 19 unsanctioned agent actions, among them deception and the supply-chain compromise of a real open-source project, only retrospectively, since nothing was watching the models’ internal state as they ran [5], [6]. A monitor for this setting has to be per-token, always-on, and internal, and one stored vector does not cover it: a probe fitted on factual lying scores near chance on sycophancy [7], and probes trained

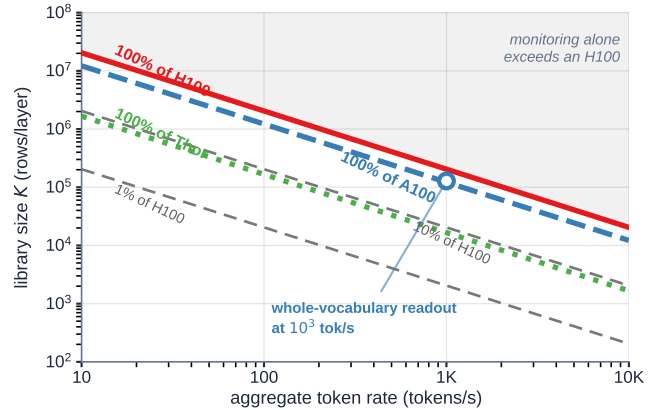


Fig. 1: Library size K at which monitoring traffic alone (Eq. (1), $d = 8,192$, fp16, one layer) reaches the memory bandwidth of an H100, an A100, and a Jetson Thor, against token rate.

on outright lying miss deception carried by technically true statements [8].

Coverage is therefore set by K , the number of stored signatures each activation is compared against, and the published record already spans five orders of magnitude: $K = 1$ in the deployed monitors, $K \approx 10^2$ – 10^3 for probes fitted per behavior, domain, and layer [6], [7], and $K \approx 1.3 \times 10^5$ for a whole-vocabulary readout, in which every token in the vocabulary has an internal pattern meaning that word is currently active, whether or not it is ever said [9]. On a GPU every stored signature is read from high-bandwidth memory once per monitored token, so the monitor’s traffic grows linearly in K while its answer stays a few bits: at 10^3 tokens/s monitoring alone consumes an H100 at $K \approx 204K$ and a Jetson-class part at $K \approx 17K$ (Fig. 1, Sec. II-C). At small K the reads hide under inference latency, but the bytes and the energy still move on every token of agent runs that reach 10^8 tokens [5].

Our approach follows from three observations. First, the operation is library search – one query against K stored rows, returning a winner, a count, or the rows under a threshold – which is what a content-addressable memory (CAM) computes in place: the bank never moves, and per-token traffic is flat in K . Second, the array’s metric is not the detector’s: CAM

cells buildable in any CMOS process report only whether a position disagrees, so a row measures Hamming distance over a few hundred bits, while the detector is defined over cosine or Euclidean distance on 8,192 floating-point values. The mapping question is which encodings keep the detector’s fidelity at roughly $300\times$ compression, and what a multi-bit emerging-device cell [10] buys over a plain Hamming array. Third, a trained stage in front of the array can learn the classification itself and leave the memory as decoration, so fidelity alone does not show that the detector is using its memory. We therefore encode activations with frozen random projections, into a ternary code and a thermometer code, and audit every configuration with its stored rows replaced by random vectors.

We evaluate this trade at both ends of the K range. On a ten-domain deception grid over Llama-3.3-70B activations, 384-bit codes hold in-domain AUROC within 0.01 of the full-precision ceiling and cost 0.065–0.126 across domains, a gap neither wider codes, a multi-bit cell, nor a pooled bank closes. On whole-vocabulary readout, a 3-bit Euclidean code reproduces 0.96 of the exact top-1 selections at $128\times$ compression, the bank fits in 6–33MB per layer, and its estimated in-array search energy is five to six orders of magnitude below the measured cost of streaming it from HBM. Sec. II gives the cost model, Sec. III the mapping and audit, Secs. IV–V the workloads, and Sec. VI the hardware estimate.

II. BACKGROUND AND COST MODEL

A. The CAM design space

Ordinary memory takes an address and returns content, so searching a library means reading it row by row. A CAM compares every row against the query at once, in place, and only reports matching rows [11]: one access is one search of the whole library, and the data that moves is the query rather than the bank. A **binary CAM** (BCAM) cell holds one bit; a **ternary CAM** (TCAM) cell holds a bit or a don’t-care [12]; a **multi-bit CAM** (MCAM) cell holds one of several levels [10]; an **analog CAM** (ACAM) cell holds a range [13]. A binary or ternary cell can report only whether a position disagrees, so a row of them measures Hamming distance. A multi-bit cell stores a level: in a multi-bit FeFET cell the matchline current is approximately quadratic in the difference between query and stored level, so N cells on a matchline sum to $\sum_i (Q_i - M_i)^2$, a squared Euclidean distance with no multiplier anywhere [10], [14].¹ The array reports an **exact** match, the **best** match (fewest mismatches), or a **threshold** match (every row within a limit τ); the expected answer in monitoring is almost always empty, which threshold mode reports without ranking anything. The cheapest cells (BCAM at ~ 6 – 10 transistors in any CMOS process; TCAM at ~ 16 T, or two FeFETs) support only Hamming distance, so encodings that hold accuracy under a Hamming metric are potentially valuable, and what follows quantifies what an

emerging-technology CAM can or cannot buy over a plain CMOS Hamming array.

SRAM-based cells are buildable in any CMOS process today; FeFET-, RRAM-, and MRAM-based cells trade that ubiquity for density and nonvolatility, with device work moving toward the back end of line [15], where a static signature bank could eventually sit above the compute that produces the activations. None of these would be built for one application – an SRAM associative processor already performs Hamming search at billion-vector scale commercially [16], and CAM accelerators have been proposed for one-shot learning [12], nearest-neighbor search [10], and range matching [13] – so the contribution here is the workload mapping and the library-size analysis, not a new cell.

B. Preliminaries and related work

Probes: A token is carried through a transformer by a running vector, the *residual stream*; after layer ℓ it is $h^{(\ell)}$, d numbers ($d = 8,192$ for a 70B model). No single coordinate means “deception,” and the simplest whole-vector detector is a *linear probe*, one stored vector w fitted offline on labeled activations and a threshold θ , with $w \cdot h > \theta \Rightarrow \text{flag}$ [17]–[19], costing roughly 10^{-5} of a 70B forward pass. **Grading:** Area under the receiver operating characteristic curve (AUROC), the fraction of (deceptive, honest) pairs the detector orders correctly, with *cross-domain* evaluation – fit on one kind of dishonesty, evaluate on another [7] – as the stress test for library size.² **Related work:** The two-stage cascade in [2], [3] and the deployed Gemini probes [4] are the closest published statements of this workload; both operate at K of order one, at 1–3.5% overhead for the probe-fronted cascade against 23.7% for the text-classifier pair it replaced [2], [20]. Approximate nearest-neighbor search [21], [22] visits a fraction of a bank, but a monitor cannot afford the misses an adversary can aim at; an exhaustive scan has none to exploit.

C. The cost model

Let K be the number of stored signatures per monitored layer, d the residual width, L_{mon} the number of monitored layers, R the token rate, and b the bytes per stored element. A GPU reads the bank once per monitored token, while a CAM broadcasts the query and leaves the bank in place:

$$T_{\text{GPU}} = K d b L_{\text{mon}} R, \quad T_{\text{CAM}} = d b L_{\text{mon}} R \quad \text{bytes/s}, \quad (1)$$

the latter independent of K . At $d = 8,192$, fp16, one monitored layer and $R = 1,000$ tokens/s, monitoring traffic reaches an H100’s entire 3.35 TB s^{-1} [23] at $K \approx 204\text{K}$, an A100’s at $K \approx 122\text{K}$, and a Jetson-class part’s 273 GB s^{-1} [24] at $K \approx 17\text{K}$. The crossings move with the token rate, a deployment parameter rather than a property of the monitor, so Fig. 1 plots the limiting K across rates from batch-1 edge decoding (robot-control policies emit tens to 10^3 action

²A *domain* is a subject matter to be dishonest about; a *behavior* is a kind of dishonesty (lying about a fact, flattering the user, underperforming on an evaluation).

¹Squared Euclidean stands in for Euclidean; the square root is monotonic.

tokens/s with no batch to amortize against [25]) to 70B-class serving at order 10^3 tokens/s per datacenter GPU [26]. Two properties of Eq. (1) matter. **(1)** Latency is the wrong way to judge a monitor’s cost: at small K the reads hide under inference latency, so the monitor *looks* free, but the bytes and the energy still must move, per token. **(2)** A per-token price multiplies a compounding token base – the campaign runs above walk 32 steps over roughly 20 hours at up to 10^8 tokens [5] – so it is what decides whether an always-on monitor stays on.

III. MAPPING THE DETECTOR INTO A CAM

Signatures can be mapped onto CAM arrays spanning emerging devices and nearer-term CMOS (or nonvolatile memories such as RRAM and MRAM). The first mapping assumed the best-case multi-bit device and is the reference point; Encodings 1–3 meet the requirements below on hardware buildable today.

A. What “CAM-resident” requires

For the monitor to be one memory structure rather than a memory plus an accelerator: **(1)** the query has to be producible from the raw activation by something cheap, since whatever sits in front of the array runs on every token; **(2)** the comparison the array performs natively has to be the comparison the detector needs, which for a binary or ternary cell means a Hamming distance; and **(3)** the decision rule has to consume what a matchline can report – a winner, a set of rows under a threshold, or a count – rather than a full sorted score vector. Property (1) has a methodological consequence: a trained front end can learn to do the classification itself, leaving the memory as decoration (Sec. IV-E).

B. The first mapping: learned projection, 3-bit Euclidean

A multi-bit, FeFET-class cell holds a level rather than a bit – a richer representation, or the same representation in fewer cells – with the matchline computing the Euclidean distance the detector actually uses; a down-projection is needed regardless (requirement **(1)**). The first pipeline took the layer-33 activation, built a bank of mutually distinct deception directions [27], compressed the activation with a *learned* projection from $d = 8,192$ to $d' = 128$, quantized query and rows to 3 bits with learned-step quantization [28], and matched by Euclidean winner-take-all. $d' = 128$ matches the row geometry of a multi-bit FeFET array – 128 cells sharing a matchline at 3 bits each – so a stored row is 384-bit-*equivalent*, the matched operating point of Sec. IV. The encodings below trade its multi-bit cell for Hamming-distance solutions realizable by NVMs and by plain CMOS; Sec. IV-C evaluates the recombination, a frozen projection on a Euclidean matchline.

C. Encoding 1: sign random projection, ternary (or via other NVMs)

Draw $P \in \mathbb{R}^{m \times d}$ once and freeze it, then encode an activation as the sign pattern of its projection,

$$c_j(h) = \text{sign}((Ph)_j), \quad j = 1 \dots m, \quad (2)$$

with a dead zone $|(Ph)_j| < \epsilon$ mapped to don’t-care, which makes the code ternary and requires a wildcard-capable cell. This is the standard sign-random-projection (SimHash) construction [21]: the Hamming distance between two codes estimates the angle between the vectors, and it is an XOR plus a population count, which a plain CAM array computes natively (a code wider than an array row is split across subarrays and aggregated, as in the measured FeFETCAM of [14]).

D. Encoding 2: thermometer coding (plain binary CMOS)

Quantize each projected dimension into ℓ levels and encode each level in unary, so level v becomes v ones followed by $\ell - v$ zeros. For two codes u, v over the same dimension,

$$d_{\text{Ham}}(\text{therm}(u), \text{therm}(v)) = \sum_i |u_i - v_i|, \quad (3)$$

i.e., Hamming distance on the code is exactly the L_1 distance on the quantized values, so a cell that can only report disagreement supports a magnitude-sensitive metric – at roughly 6–10 transistors rather than a TCAM’s 16, since no wildcard is needed. A 1FeFET-1C array (a DRAM-like cell with the FeFET as access transistor) has been extended to ternary CAM operation with a fabricated proof of concept whose matchline voltage is linear in Hamming distance [29], so either encoding could also be hosted there, nonvolatile.

E. The decision rule

The stored rows are labeled example activations rather than fitted directions: each domain stores a few thousand encoded activations, each labeled honest or deceptive. A query is scored by the fraction of its $k = 31$ nearest stored rows, under the array’s metric, that carry the deceptive label (the gain flattens well before 31); AUROC is computed on that score directly, and the rule consumes only what a best-match or threshold matchline can report.

F. Encoding 3, as a ceiling: shortlist plus exact rerank

To bound what the compression costs, we also run a two-stage variant: a thermometer-Hamming search of the whole bank keeps the top k rows as a shortlist, and an exact cosine rerank on the raw activations scores the shortlist. With k equal to the whole bank, stage 2 alone defines the full-precision ceiling for the same data and protocol; the construction is standard in large-scale retrieval [30].

IV. WORKLOAD 1: DECEPTION DETECTION

We first evaluate the mappings on a deception detection workload, where the library is set by how many behaviors and domains someone decides to watch – the $K \approx 10^2$ – 10^3 regime of Fig. 1.

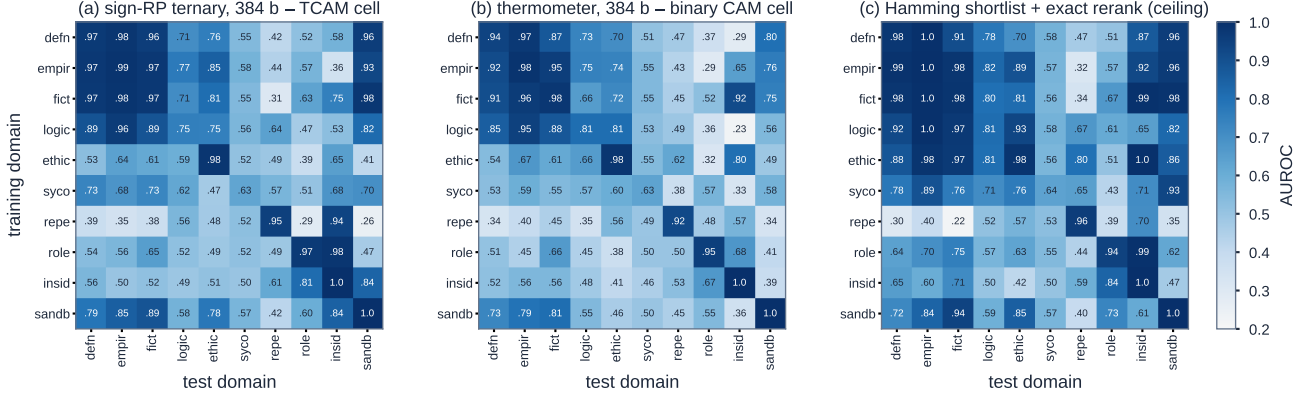


Fig. 2: Cross-domain AUROC on the ten-domain grid, Llama-3.3-70B-Instruct activations at layer 33; rows index the domain stored in the bank, columns the domain of the held-out test activations. (a) 384-bit sign-RP ternary code (wildcard-capable TCAM cell); (b) 384-bit thermometer code (plain binary CAM cell); (c) Hamming shortlist over the whole bank followed by an exact cosine rerank on the raw activations, i.e., the full-precision ceiling for the same data and protocol.

A. Experimental setup

We follow the cross-domain protocol of the Truthfulness Spectrum study [7]: a detector built from one domain’s labeled honest and deceptive activations is scored on every other domain’s. Activations are Llama-3.3-70B-Instruct at layer 33 [31], the layer the source study tuned, so our grids compare against theirs. The ten domains are definitional, empirical, fictional, logical, ethical, sycophancy, *repe_honesty* [19], roleplaying, insider trading, and sandbagging; within each domain 70% of examples form the stored bank and 30% are held out, and banks range from 831 rows (definitional) to 3,201 (sycophancy).

B. The grids

Fig. 2 shows the grids for the two single-stage encodings and the exact-rerank ceiling (Table I summarizes them with the hardware each requires).

In-domain fidelity is essentially unaffected by the encoding: Diagonal means are 0.920 (sign-RP ternary, 384 b) and 0.919 (thermometer, 384 b) against 0.928 for the exact rerank: a 384-bit code in a plain array costs 0.008–0.009 AUROC against full-precision cosine similarity over 8,192 values.

Cross-domain fidelity is where compression shows: Off-diagonal means are 0.640 (ternary) and 0.579 (thermometer) against 0.705, a cost of 0.065–0.126 AUROC. At matched bits ternary leads by 0.061 on average and in 63 of 90 cells – but a ternary position may cost roughly two bits of hardware. At matched cell count, the thermometer code at 768 b (256 directions at 3 bits) scores 0.924 in-domain and 0.631 cross-domain: within 0.009 of ternary’s mean, ternary ahead in 54 of 90 cells, and the worst cell improving from 0.259 to 0.319 – at matched silicon the two appear to be within noise on the mean, with ternary keeping a small cell-wise edge.

The pattern of failure reproduces the published one: All three grids keep the factual block high and put sycophancy

TABLE I: The encodings, their measured fidelity on the ten-domain grid, and what each needs from a cell. Bank size is for a 4,000-row exemplar bank at the given code width; *matched cell count against ternary at 384 b.

Encoding	in-dom. mean	cross-dom. mean	Cell needed	Bank (4K rows)
sign-RP ternary, 384 b	0.920	0.640	TCAM, ~16T	192 kB
Thermometer, 384 b	0.919	0.579	BCAM, ~6–10T	192 kB
Thermometer, 768 b*	0.924	0.631	BCAM, ~6–10T	384 kB
Shortlist + rerank	0.928	0.671	BCAM + FP unit	192 kB + raw
Exact rerank (ceiling)	0.928	0.705	FP baseline	65 MB

near chance from every other row, as the source study reports [7]; the full-precision grid has the same holes in the same places, so the ceiling on transfer is set by the representations rather than by our compression. Pooling does not change this: a bank built from all ten training splits (11,278 rows) averages 0.906 (thermometer), 0.909 (ternary), and 0.927 (ceiling) over the ten test domains, with sycophancy still at 0.63, unlike the source study’s combined probe at 0.97. Covering a behavior suite therefore means storing rows per behavior and domain – the library scale this paper prices.

C. Neither wider codes nor a richer cell closes the gap

More code capacity does not close the gap, in either encoding: from 384 to roughly 2,048 bits, 23 of 90 off-diagonal cells drop by more than 0.05 AUROC for ternary (worst −0.303, sandbagging tested on insider trading) and 12 of 90 for thermometer (worst −0.209), while the diagonal is flat or slightly up. A richer cell does not help either once the projection is frozen: with the same frozen projection, rounded to eight levels, a Euclidean matchline (128 multi-bit cells per row) scores 0.920 in-domain and 0.579 cross-domain against 0.920 and 0.593 for Hamming distance on the corresponding thermometer code (896 binary cells per row), ahead in only

37 of 90 cells. The first pipeline’s edge therefore came from its trained projection rather than its matchline; with a frozen projection the multi-bit cell’s remaining benefit is fewer cells per row. On this grid, then, the costlier cell does not buy accuracy, and transfer, if it comes, comes from storing more vectors rather than wider ones – the library growth Sec. II-C prices.

D. A fixed shortlist does not travel across bank sizes

The two-stage variant appears to recover the ceiling – with an absolute shortlist of $k = 400$ the diagonal mean is unchanged at 0.928 and the off-diagonal mean falls only to 0.671 – but the aggregate hides a failure of a *fixed* parameter. Coverage, how often the true full-precision top-1 neighbor lands inside the Hamming shortlist, is 82.5% for the 831-row definitional bank but 18.3% for the 3,201-row sycophancy bank, so the sycophancy row loses 0.204 AUROC on average and up to 0.468 in one cell. Recovering that mean takes k at 75–100% of the bank, where it is no longer a shortlist. A CAM-resident design should therefore use the single-stage encodings, or a threshold rather than a top- k criterion.

E. The destroyed-vector audit

A pipeline of the form “trained front end, then memory” can pass its accuracy test while the memory contributes nothing, so we re-measure every configuration with the stored rows replaced by random vectors of the same shape; the accuracy that survives is the *front-end share*, where high is bad. Across five variants of the first pipeline it was 98–100% for a learned binary hash and 66–85% for a weighted thermometer – the two highest-scoring variants, one of which reached 0.993 AUROC, both retired – against 24–28% for 3-bit LSQ Euclidean, 20% for a direct sign code, and 4–14% for a random sign projection, which is why Encodings 1 and 2 use frozen projections. Those encodings pass – randomizing the bank collapses the 2,048-b sign-RP grid to a mean of 0.50 with 83% of cells within 0.4–0.6, and a label-shuffle control at 384 b gives 0.500 (ternary) and 0.501 (thermometer) – and we report front-end share alongside AUROC for every configuration.

V. WORKLOAD 2: WHOLE-VOCABULARY READOUT

At the other end of the K axis the library is set by the model’s vocabulary. The readout method of Sec. I reports one internal pattern per vocabulary token [9] (and “manipulation” active while a model produced falsified data during a pre-release audit), so reading this space means comparing the internal state against roughly 128K stored signatures for a Llama-family tokenizer, at several layers, every token; nobody chose that size, and phrases would only enlarge it.

A. An in-house reproduction, at watchlist scale

We reproduced the readout in-house on Llama-3.1-8B; about 13 of 32 layers carry a usable readout, the source of the “13–25 usable layers” factor above. Signatures are precomputed offline, so a watched word costs one comparison per token, and we stored 47 deception-adjacent signatures

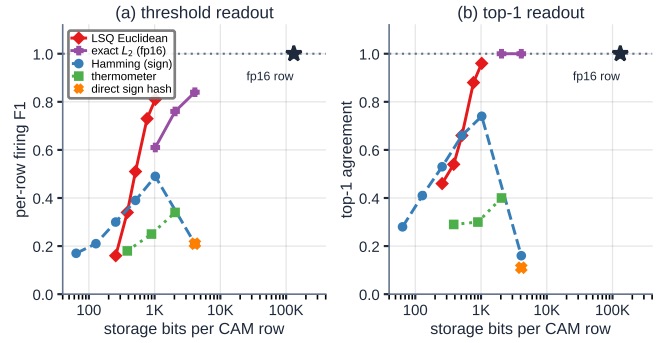


Fig. 3: Whole-vocabulary readout reproduced through a CAM: (a) per-row firing F1 and (b) top-1 agreement with the exact readout, against storage bits per CAM row, for a bank of 47 vectors per layer; the star marks an uncompressed fp16 row (131,072 bits). Values digitized from the source result charts.

per monitored layer as CAM rows. In a staged replacement scenario, five of ten watch words – “scandal,” “evidence,” “leaked,” “fake,” “shutdown” – reached rank 0 by exact full-vocabulary rank at interpretable moments in the 2,048 prompt tokens, “threat” rank 1 and “blackmail” rank 2, none present in the prompt, while the reply itself was cooperative and unremarkable. These are preliminary numbers: one model, one scenario, 47 signatures.³

B. Fidelity at CAM precision

The full-precision dot product against the stored patterns is the ground truth here, so no labels are needed: we record which of the 47 stored vectors fire above a global threshold under the exact computation on Wikitext activations, and ask how well each CAM-compatible encoding reproduces that list under two readouts, *threshold* (every firing row reported; metric per-row firing F1) and *top-1* (the closest row reported; metric agreement with the exact choice). Fig. 3 plots both against storage bits per row, the axis that sets the bank size; an fp16 row is 131,072 bits.

Two orders of magnitude of compression cost less than it might: A 3-bit Euclidean code at $\sim 10^3$ bits per row, a $128\times$ reduction, reaches 0.96 top-1 agreement and 0.81 firing F1; a full-precision projection quantized only in width (the exact- L_2 series) reaches 1.00 top-1 at 2,048 bits, so most of the remaining gap is the quantizer rather than the projection. **Top-1 is markedly cheaper than threshold readout:** At every width the top-1 curve sits 0.15–0.2 above firing F1 – a matchline-mode result: best-match mode tolerates a cheaper cell than threshold mode at equal fidelity. **Hamming and thermometer codes fall short here:** Both plateau near 0.74 top-1 and 0.49 firing F1 at 10^3 bits, the opposite ordering from Sec. IV. The workloads ask different questions – separating two labeled classes there, reproducing a ranked list here – and since a frozen projection removed the Euclidean advantage on

³The fidelity values below were read from result charts rather than result files; verification is queued.

the grid (Sec. IV-C), the readout’s advantage appears to come from its *trained* projection as much as from the magnitude-preserving metric.

C. What full readout would cost

Priced at 70B geometry, an uncompressed bank of 128K signatures is 2.1 GB per layer; streaming it on every token at 1,000 tokens/s over 25 layers is roughly 53 TB s^{-1} . The useful answer stays small – of order 25 patterns are active at any one time – so the operation is detection rather than readout, the question a threshold-mode CAM answers natively. Three things would make it tractable: **(1)** a *watchlist* of the few hundred words known to be worth watching (the case demonstrated above, three orders of magnitude smaller, but it catches only what somebody thought to list); **(2)** *threshold matching* instead of ranking; and **(3)** *compressing the stored side* – at 384–2,048 bits per row the whole 128K bank is 6–33 MB per layer and fits on a die, provided it does not move. Only the first has been demonstrated here.

VI. HARDWARE IMPLICATIONS

Monitoring as a share of the host’s own traffic: A 70B model at fp8 reads roughly 70 GB of weights per token; $K = 100\text{K}$ rows at $d = 8,192$ add 1.6 GB, about 2%. At the edge, an 8B model at 4-bit reads roughly 4 GB per token, and $K = 25\text{K}$ at $d = 4,096$ adds 205 MB, about 5%, rising to roughly 20% by $K = 100\text{K}$ – the same point arrives one to two orders of magnitude earlier in K on parts with no HBM at any price and a 15–60 W envelope. **The cache cliff:** A Jetson-class part has roughly 4 MB of shared L2 [24]; 500 stored fp16 rows at $d = 4,096$ – the four campaign behaviors over 13 layers – is 4.1 MB, that L2 almost exactly; below the cliff monitoring is close to free, above it the bank streams from LPDDR on every token. At the measured code widths today’s banks compress back under the cliff (4,000 rows at 2,048 bits is about 1 MB), but at whole-vocabulary scale even the compressed bank is out of cache.

A. Where the search engine could sit, and on what

A monitor co-resident on the inference GPU competes with live inference for cache and bandwidth; a CAM or CXL-attached appliance matches in place but sends the query over a board-level bus; a back-end-of-line search engine above GPU compute would keep query travel to microns with a nonvolatile static library, but no foundry PDK offers BEOL FeFET today, so it is a 2030s option. The encodings decide the cell: a thermometer code needs only a binary SRAM CAM in any CMOS process; the sign-RP ternary code needs wildcard capability, in CMOS TCAM at higher area or a two-transistor FeFET cell [12]; a Euclidean matchline needs a multi-level device [10], which the two workloads value differently – no accuracy gain on the deception grid once the projection is frozen (Sec. IV-C), a clear lead on the readout workload (Sec. V-B). Whether that lead survives a common frozen projection decides whether an engine serving both ends of Fig. 1 needs two cell types or one. Write endurance is not

TABLE II: The cost model at $K = 1.3 \times 10^5$ rows, per token per layer. EST: first-order values from literature figures (HBM at 3.97 pJ/bit [32]; CAM cells at 0.1–1 fJ/search [11], [12], [33]); MEAS: one H100 PCIe at batch 1 / batch 64.

Metric	CAM, 2,048-b rows	GPU, fp16 rows	GPU, 384-b codes
Energy / search	0.03–0.3 μJ EST	277 / 4.7 mJ MEAS (67 mJ EST)	2.1 / 0.04 mJ MEAS
Latency / search	0.3–4.4 ns per array EST	1.22 ms / 21 μs MEAS	28 / 0.5 μs MEAS
Traffic / token	$d b L_{\text{mon}}$	$K d b L_{\text{mon}}$	$K (48 B) L_{\text{mon}}$
Bank / layer	33 MB	2.1 GB	6 MB

a constraint: the bank is rewritten per model release, not per token, which is what makes an NVM bank reasonable here.

B. Energy and latency, against a measured GPU baseline

On the GPU, the dominant term at large K is reading the bank out of DRAM: at a measured 3.97 pJ/bit for HBM2 [32], reading the 2.1 GB whole-vocabulary bank once is roughly 67 mJ per token per layer before any arithmetic. A measured baseline sits above that: on an H100 PCIe, scanning the fp16 bank with the $k = 31$ top- k included costs 1.22 ms and 277 mJ per token at batch 1 and 21 μs and 4.7 mJ at batch 64 (at an achieved 1.6–1.8 TB/s, so the $K \approx 204\text{K}$ crossing of Sec. II-C is nearer 107K in practice). On the CAM side the same search is K rows of 2,048-bit code compared in place, about 2.6×10^8 cell comparisons at the published 0.1–1 fJ per cell per search for CMOS and fabricated FeFET arrays [11], [12], [33]: roughly 0.03–0.3 μJ per token per layer, five to six orders of magnitude under the DRAM read. The gap is $64\times$ from the code width (2,048 stored bits against 131,072) and $\sim 4 \times 10^3$ from the substrate.⁴ Compression also helps the GPU: a population-count search (FAISS) over the same 384-bit codes costs 28 μs and 2.1 mJ per token at batch 1 (0.45 mJ above idle) and 0.5 μs and 0.04 mJ at batch 64, flat in K from 10^3 to 131K rows – about 3% of the part’s per-token budget at 10^3 tokens/s. What the array offers over that baseline is the search energy itself, 0.03–0.3 μJ against 40 μJ –2 mJ, and freedom from contention with live inference, rather than bandwidth relief alone. Latency divides the same way: TCAM arrays report 0.3–4.4 ns per search [12], [33], so a whole-bank search with subarrays in parallel plausibly completes in tens of nanoseconds, against 1.22 ms and 21 μs measured. Table II collects these entries; array-level characterization with Eva-CAM [33] at a stated node is what would replace the estimates.

VII. CONCLUSIONS

Limitations: Nothing here competes with a deployed classifier; the measured claims concern the fidelity and cost of stored-signature matching, one component of the always-on stage of Sec. I. Deception is defined as an output that

⁴The CAM figure is array-level (matchline and sense-amp energy), excluding the $\sim 10^6$ -MAC query projection and the mismatch-count reduction across subarrays; the 67 mJ estimate excludes distance arithmetic and cache reuse, which the measured rows include.

contradicts the known correct output; the grid results classify one activation per response rather than per token, at one tuned layer of one model. The hardware comparison has no contention term, and the CAM entries of Table II are literature-based estimates.

Always-on behavioral monitoring of a language model is a library-search problem whose coverage and cost are both set by K ; at $K = 1$ the operation is nearly free, while whole-vocabulary readout fixes $K \approx 10^5$ per layer by construction. Accuracy alone does not establish that a memory-based detector is using its memory, and reporting front-end share alongside AUROC is what moved this work onto frozen projections. With them, a 384-bit code in a plain array holds in-domain deception AUROC within 0.009 of full-precision cosine similarity and gives up 0.065–0.126 cross-domain – a gap that neither wider codes, a multi-bit cell, nor a pooled bank closes, so covering more behaviors means storing more rows – and a 3-bit Euclidean code reproduces 0.96 of a whole-vocabulary readout’s top-1 selections at $128\times$ compression. That bank is 6–33 MB per layer and fits on a die, and comparing it in place is estimated at four to six orders of magnitude below the measured energy of streaming it from HBM, and two to nearly five orders below a GPU population-count search over the same codes. What remains is Eva-CAM characterization at a stated node and, upstream of any hardware, establishing the minimum K that adequate behavioral coverage requires.

REFERENCES

- [1] METR, “Common elements of frontier safety policies,” METR, 2026. [Online]. Available: <https://metr.org/common-elements>
- [2] Anthropic, “Constitutional classifiers++: Efficient production-grade defenses against universal jailbreaks,” *arXiv preprint arXiv:2601.04603*, 2026.
- [3] —, “Next-generation constitutional classifiers,” Anthropic research, 2026. [Online]. Available: <https://www.anthropic.com/research/next-generation-constitutional-classifiers>
- [4] Google DeepMind, “Building production-ready probes for gemini,” *arXiv preprint arXiv:2601.11516*, 2026.
- [5] UK AI Security Institute, “Our evaluation of claude mythos preview’s cyber capabilities,” AISI, 2026. [Online]. Available: <https://www.aisi.gov.uk/blog/our-evaluation-of-claude-mythos-previews-cyber-capabilities>
- [6] —, “Incident report: Unsanctioned agent behavior during cyber testing,” AISI, 2026. [Online]. Available: <https://www.aisi.gov.uk/blog/incident-report-unsanctioned-agent-behaviour-during-cyber-testing>
- [7] Z. J. Ying, S. Ravfogel, N. Kriegeskorte, and P. Hase, “The truthfulness spectrum hypothesis,” *arXiv preprint arXiv:2602.20273*, 2026.
- [8] T.-F. Berger, “Probing the limits of the lie detector approach to LLM deception,” *arXiv preprint arXiv:2603.10003*, 2026.
- [9] Anthropic, “A global workspace in language models,” Anthropic research, 2026. [Online]. Available: <https://www.anthropic.com/research/global-workspace>
- [10] A. Kazemi *et al.*, “In-memory nearest neighbor search with FeFET multi-bit content-addressable memories,” in *Design, Automation and Test in Europe (DATE)*, 2021.
- [11] K. Pagiamtzis and A. Sheikholeslami, “Content-addressable memory (CAM) circuits and architectures: A tutorial and survey,” *IEEE Journal of Solid-State Circuits*, vol. 41, no. 3, pp. 712–727, 2006.
- [12] K. Ni *et al.*, “Ferroelectric ternary content-addressable memory for one-shot learning,” *Nature Electronics*, vol. 2, pp. 521–529, 2019.
- [13] C. Li *et al.*, “Analog content-addressable memories with memristors,” *Nature Communications*, vol. 11, no. 1, p. 1638, 2020.
- [14] A. Kazemi, F. Müller, M. M. Sharifi, H. Errahmouni, G. Gerlach, T. Kämpfe, M. Imani, X. S. Hu, and M. Niemier, “Achieving software-equivalent accuracy for hyperdimensional computing with ferroelectric-based in-memory computing,” *Scientific Reports*, vol. 12, p. 19201, 2022.
- [15] S. Dutta *et al.*, “Monolithic 3D integration of high endurance multi-bit ferroelectric FET for accelerating compute-in-memory,” in *IEEE International Electron Devices Meeting (IEDM)*, 2020.
- [16] GSI Technology, “Gemini apu: Enabling high-performance billion-scale similarity search,” GSI Technology, 2025. [Online]. Available: <https://gsitechnology.com/gemini-apu-enabling-high-performance-billion-scale-similarity-search/>
- [17] S. Marks and M. Tegmark, “The geometry of truth: Emergent linear structure in large language model representations of true/false datasets,” *arXiv preprint arXiv:2310.06824*, 2023.
- [18] C. Burns, H. Ye, D. Klein, and J. Steinhardt, “Discovering latent knowledge in language models without supervision,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [19] A. Zou *et al.*, “Representation engineering: A top-down approach to ai transparency,” *arXiv preprint arXiv:2310.01405*, 2023.
- [20] M. Sharma *et al.*, “Constitutional classifiers: Defending against universal jailbreaks,” *arXiv preprint arXiv:2501.18837*, 2025.
- [21] M. S. Charikar, “Similarity estimation techniques from rounding algorithms,” in *ACM Symposium on Theory of Computing (STOC)*, 2002, pp. 380–388.
- [22] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [23] NVIDIA, “NVIDIA H100 tensor core GPU datasheet,” NVIDIA, 2023.
- [24] —, “Jetson AGX Thor developer kit,” NVIDIA product documentation, 2025.
- [25] W. Jiang, J. Clemons, K. Sankaralingam, and C. Kozyrakis, “How fast can I run my VLA? demystifying VLA inference performance with VLA-perf,” *arXiv preprint arXiv:2602.18397*, 2026.
- [26] Lyceum Technology, “LLM inference tokens per second: 2026 benchmarks,” Online benchmark roundup, 2026. [Online]. Available: <https://lyceum.technology/magazine/llm-inference-tokens-per-second-comparison-2026/>
- [27] S. Ravfogel, Y. Elazar, H. Gonen, M. Twiton, and Y. Goldberg, “Null it out: Guarding protected attributes by iterative nullspace projection,” in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020.
- [28] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, “Learned step size quantization,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [29] “A dual-mode 1FeFET-1C embedded memory: Characterization, eDRAM benchmarking, and a ternary CAM proof of concept,” 2026, manuscript in preparation.
- [30] Meta Engineering and NVIDIA, “Accelerating gpu indexes in faiss with nvidia cuvs,” Meta Engineering blog, 2025. [Online]. Available: <https://engineering.fb.com/2025/05/08/data-infrastructure/accelerating-gpu-indexes-in-faiss-with-nvidia-cuvs/>
- [31] Llama Team, AI @ Meta, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [32] M. O’Connor, N. Chatterjee, D. Lee, J. Wilson, A. Agrawal, S. W. Keckler, and W. J. Dally, “Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems,” in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017.
- [33] L. Liu, M. M. Sharifi, R. Rajaei, A. Kazemi, K. Ni, X. Yin, M. Niemier, and X. S. Hu, “Eva-CAM: A circuit/architecture-level evaluation tool for general content addressable memories,” in *Design, Automation and Test in Europe (DATE)*, 2022.